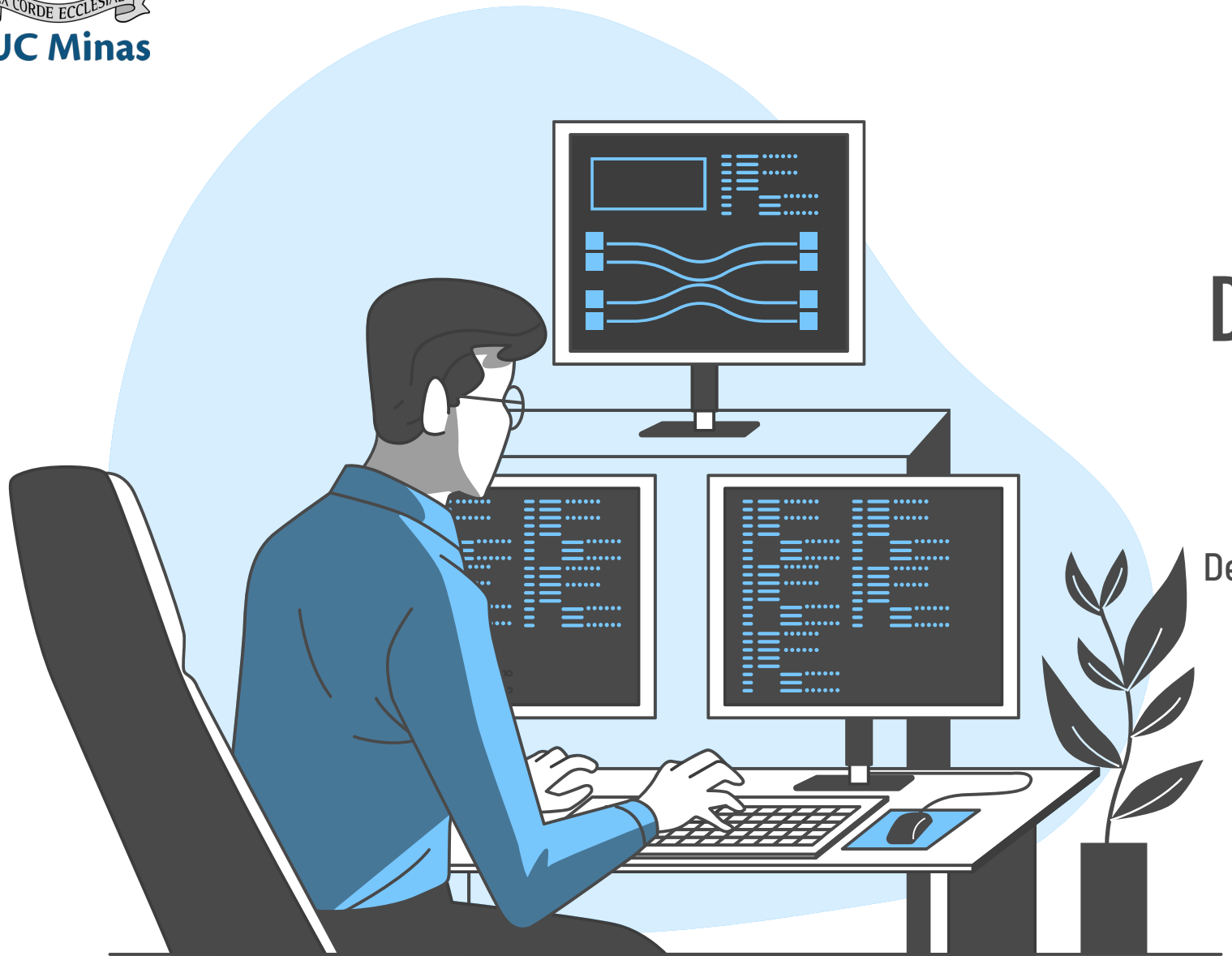
PUC Minas



Aula 06 · Deploy

Deploy do Spring Boot no Render

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Criar um **Web Service** no Render a partir do GitHub

2

Configurar **Language: Docker**, **Root Directory** e **Dockerfile Path**

3

Cadastrar variáveis: `MONGODB_URI` ou `DB_*`, `CORS_ORIGINS`, `JWT_SECRET`

4

Entender a variável **PORT** e o **health check**

5

Ler os **logs**, lidar com o *spin down* do plano gratuito e usar o `render.yaml`

Antes de começar

- Aula 03: Dockerfile do back testado localmente
- Banco na nuvem (Atlas com `0.0.0.0/0` ou Supabase *Session pooler*)
- Conta no Render (login com GitHub)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Passo 1: novo Web Service

02 Passo 2: conectar o repositório

03 Passo 3: configurar o serviço

04 Passo 4: variáveis de ambiente

05 Passo 5: acompanhar o deploy

06 O plano gratuito dorme

07 Health check

08 Blueprint: render.yaml

09 Erros comuns no Render

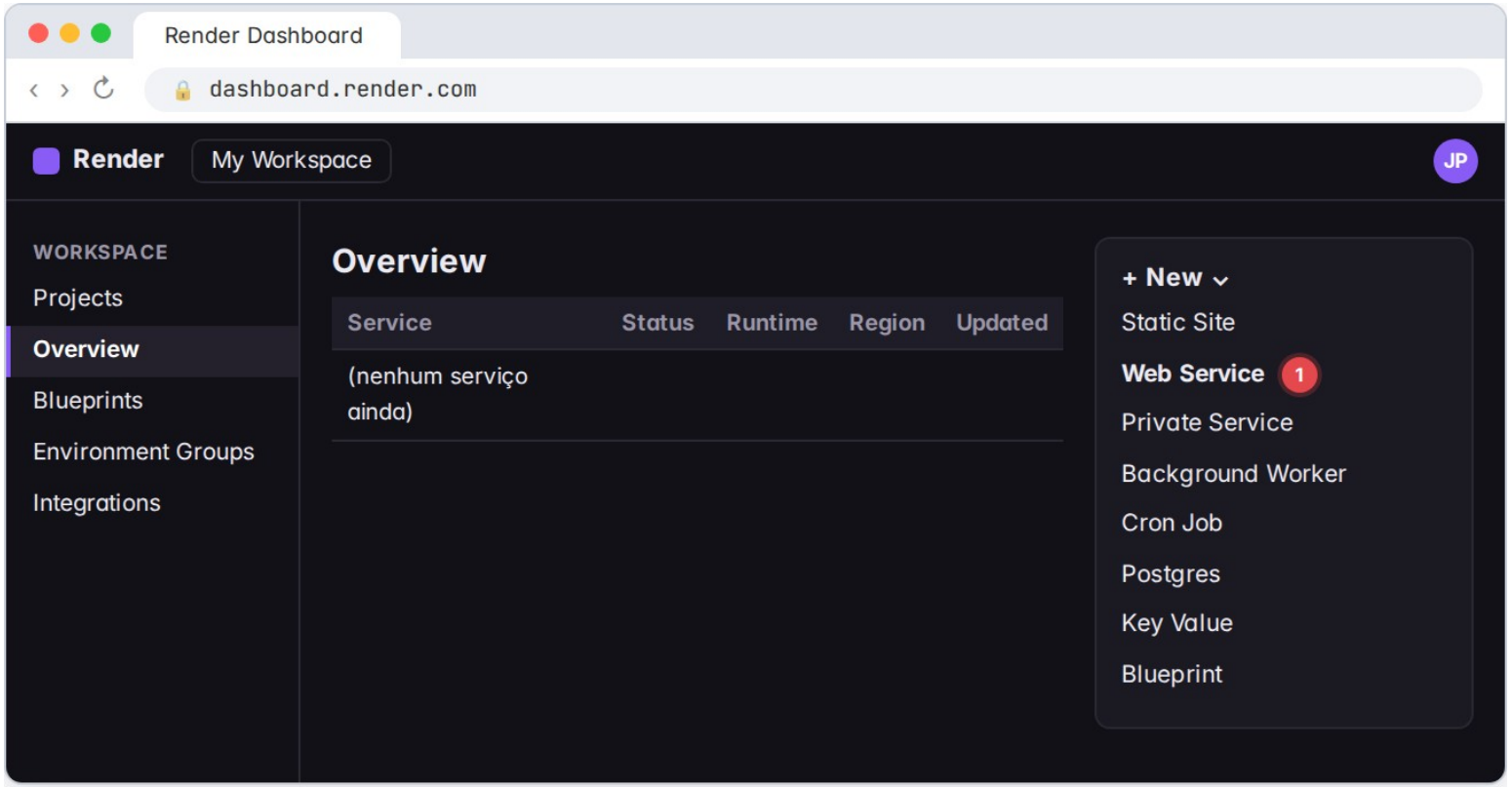
10 Aprofundando: Render além do básico

01

Passo 1: novo Web Service

Passo 1: novo Web Service

No painel do Render clique em **+ New** e escolha **Web Service** (um processo que fica ligado e responde HTTP).

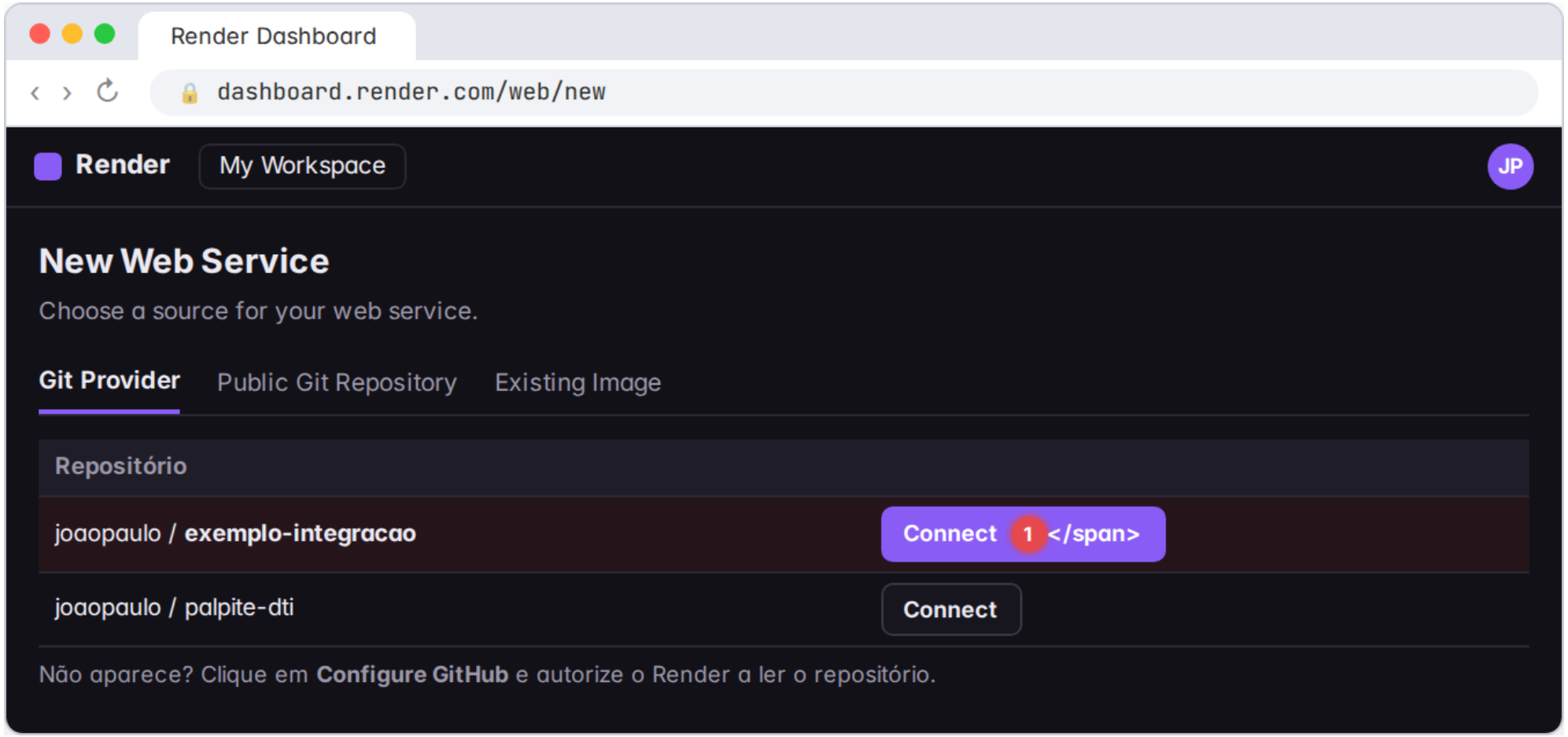


Menu **+ New**: escolha **Web Service**.

02

Passo 2: conectar o repositório

Passo 2: conectar o repositório

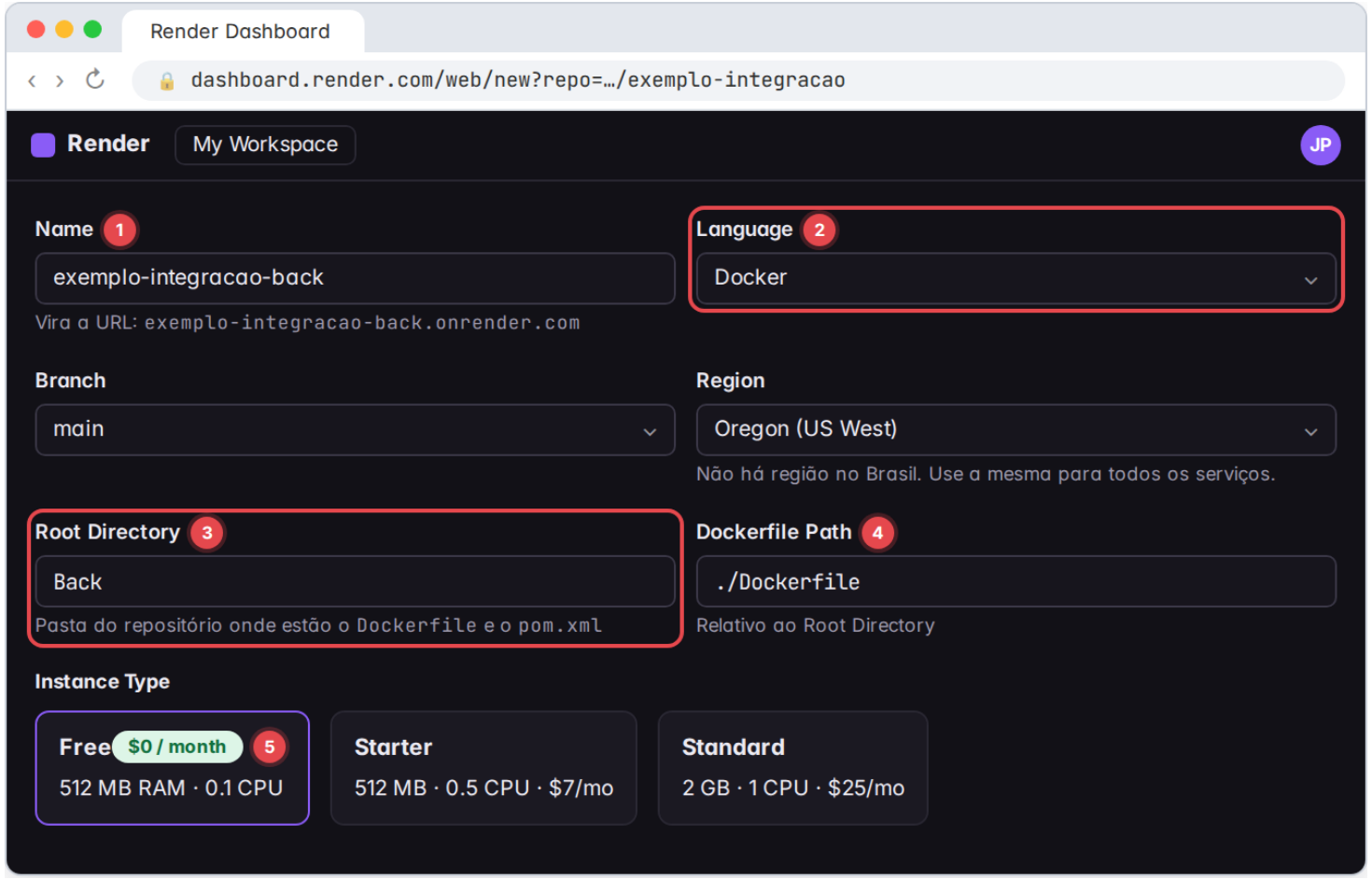


Escolha o repositório que contém a pasta `Back`.

03

Passo 3: configurar o serviço

Passo 3: configurar o serviço



Configuração do Web Service. Com **Language: Docker**, o Render usa o seu `Dockerfile`; não pergunta comando de build nem de start.

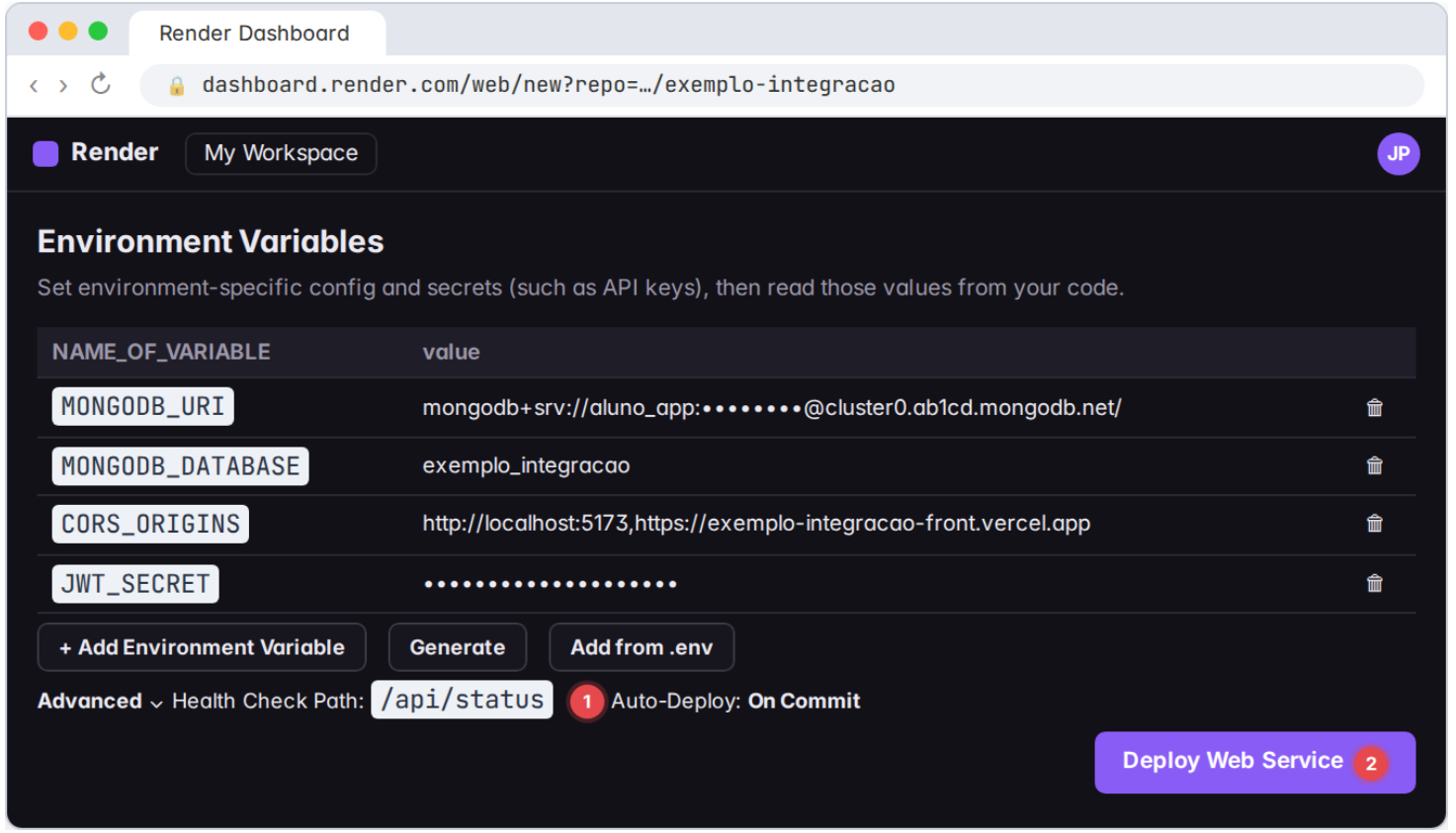
Passo 3: configurar o serviço

1. **Name**: define a URL pública do back.
2. **Language: Docker**. O Render não tem suporte nativo a Java; com Docker ele roda qualquer coisa.
3. **Root Directory: `Back`**. Sem isso o Render procura o Dockerfile na raiz do repositório e falha.
4. **Dockerfile Path**: `./Dockerfile` (dentro de `Back`).
5. **Instance Type: Free**.

04

Passo 4: variáveis de ambiente

Passo 4: variáveis de ambiente



Variáveis lidas pelo `application.properties` via `${MONGODB_URI}`, `${CORS_ORIGINS}` etc.

Passo 4: variáveis de ambiente

Para projetos com **Supabase**, troque `MONGODB_*` por:

```
Variáveis para Supabase (Session pooler, IPv4)
DB_URL=jdbc:postgresql://aws-0-sa-east-1.pooler.supabase.com:5432/postgres?sslmode=require
DB_USER=postgres.abcdefghijklmnop
DB_PASSWORD=sua-senha-do-banco
```

Passo 4: variáveis de ambiente

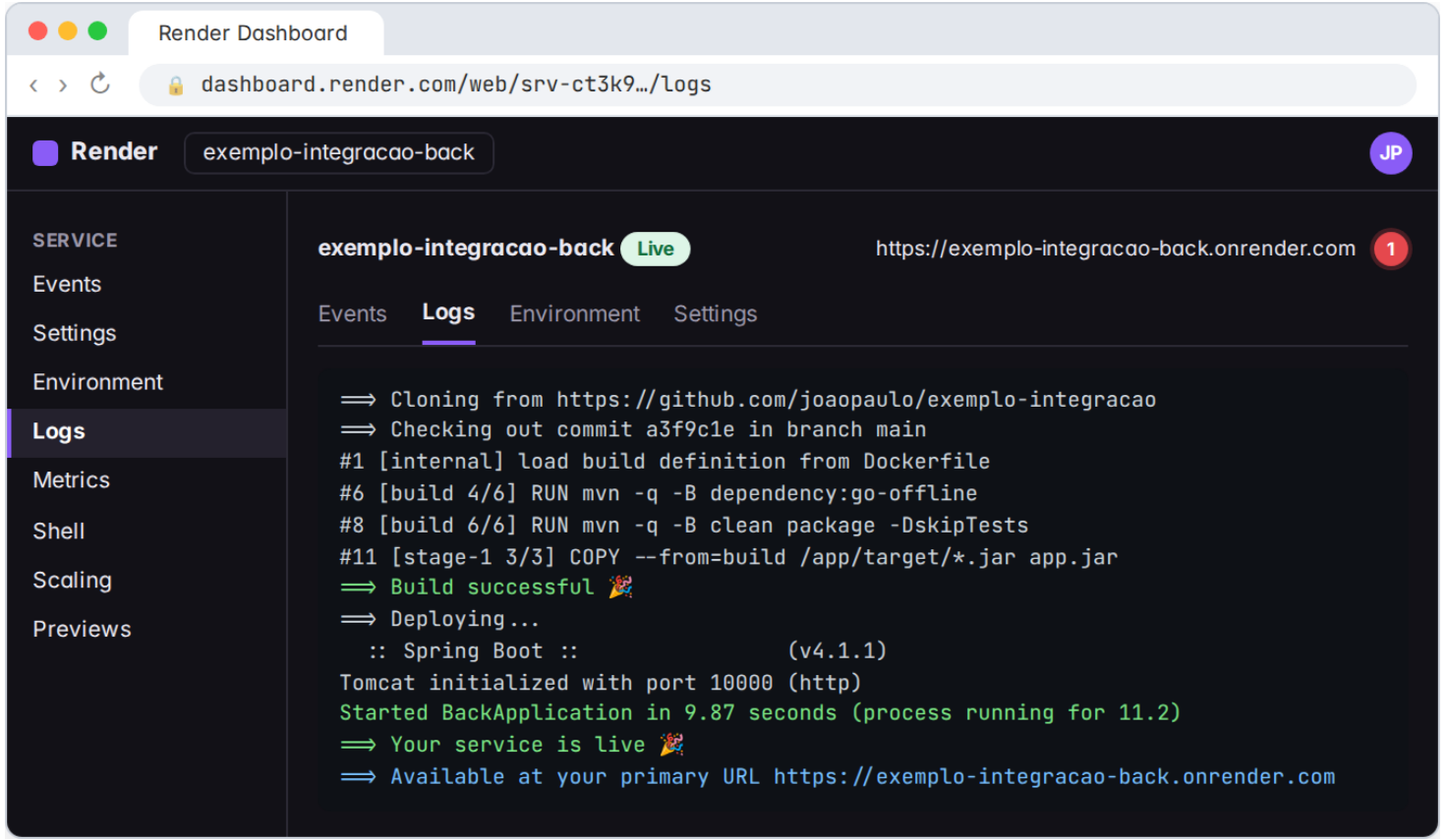
Observação

A variável PORT. O Render define `PORT` (por padrão `10000`) e espera que a aplicação escute nela. Com `server.port=${PORT:8080}` no `application.properties`, o Spring usa a do Render em produção e a 8080 no seu computador. O Render também detecta a porta aberta automaticamente, mas não conte com isso: use a variável.

05

Passo 5: acompanhar o deploy

Passo 5: acompanhar o deploy



Aba **Logs**: build do Docker, inicialização do Spring na porta 10000 e *"Your service is live"*.

Passo 5: acompanhar o deploy



Teste imediato: o endpoint de status responde pela internet, com o MongoDB Atlas conectado.

Passo 5: acompanhar o deploy

O primeiro deploy demora (de 3 a 8 minutos) porque baixa todas as dependências do Maven. Os seguintes são mais rápidos graças ao cache de camadas.

05

Passo 5: acompanhar o deploy

Aula 06 · Deploy

06

O plano gratuito dorme

O plano gratuito dorme

Sem requisições por **15 minutos**, o serviço gratuito é desligado. A próxima requisição o acorda, o que leva cerca de **50 segundos** (o Render mostra uma página de “Service waking up”; uma chamada `fetch` simplesmente demora). É por isso que o front dos projetos verifica `/api/status` repetidamente (FullStack, aula 04).

06

O plano gratuito dorme

Aula 06 · Deploy

07

Health check

Health check

Em **Settings → Health Check Path**, informe um endpoint leve que devolve `200`, como `/api/status` ou `/actuator/health`. O Render só direciona tráfego para a nova versão depois que ela responde. Se a nova versão não subir, a anterior continua no ar (*zero-downtime deploy*).

07

Health check

Aula 06 · Deploy

08

Blueprint: render.yaml

Blueprint: render.yaml

Em vez de clicar em tudo, descreva o serviço em um arquivo na **raiz** do repositório e use **+ New → Blueprint**:

render.yaml (baseado no PalpiteDTI)

```yaml
services:
  - type: web
    name: exemplo-integracao-back
    runtime: docker
    rootDir: Back
    dockerfilePath: ./Dockerfile
    plan: free
    region: oregon
    healthCheckPath: /api/status
    envVars:
      - key: MONGODB_URI
        sync: false            # o Render pede o valor na criação (segredo)
      - key: MONGODB_DATABASE
        value: exemplo_integracao
      - key: CORS_ORIGINS
        value: https://exemplo-integracao-front.vercel.app,https://*.vercel.app
      - key: JWT_SECRET
        generateValue: true    # o Render gera um valor aleatório
```

09

Erros comuns no Render

Erros comuns no Render

Log / sintoma	Causa	Solução
`failed to read dockerfile: open Dockerfile: no such file`	Root Directory vazio	*Settings* → *Root Directory* = `Back`
`release version 25 not supported`	Imagem de build antiga no Dockerfile	`maven:3.9-eclipse-temurin-25` (aula 03)
`No open ports detected` / *Timed out*	Spring não escuta em `PORT` ou demora demais	`server.port=${PORT:8080}`; confira erros anteriores no log
`MongoTimeoutException` / `Timed out after 5000 ms`	Atlas sem `0.0.0.0/0` no Network Access	Libere o acesso (Back-end, aula 07)
`UnknownHostException db.xxx.supabase.co`	*Direct connection* do Supabase (IPv6)	Use o **Session pooler**
`Ran out of memory (used over 512MB)`	Memória do plano Free	`-XX:MaxRAMPercentage=75`; evite carregar muito na inicialização
Primeiro acesso leva ~50 s	*Spin down* do plano Free	Esperado; acorde antes de apresentar

10

Aprofundando: Render além do básico

Tipos de serviço do Render

Web Service

Processo HTTP público: o back Spring Boot.

Static Site

Arquivos estáticos (alternativa ao Vercel para o front).

Private Service

Só acessível por outros serviços do Render.

Postgres / Key Value

Bancos gerenciados pelo próprio Render.

Aprofundando: Render além do básico

Aba do serviço	Para que serve
Events	Histórico de deploys, com botão de **Rollback**
Logs	Saída do container em tempo real, com busca
Environment	Variáveis e *secret files*
Settings	Root Directory, Dockerfile, Health Check, Auto-Deploy, domínios
Metrics	CPU, memória e requisições
Shell	Terminal dentro do container (planos pagos)

Environment Group x variáveis do serviço

Variáveis do serviço

- Valem só para aquele serviço
- Simples
- Bom para um back só

vs

Environment Group

- Conjunto reutilizado por vários serviços
- Atualiza tudo de uma vez
- Bom quando há back, worker e cron usando o mesmo banco

Aprofundando: Render além do básico

Formas de disparar um deploy

```
# Settings → Build & Deploy
Auto-Deploy: After CI Checks Pass   # espera o GitHub Actions (aula 09)

# Settings → Deploy Hook
https://api.render.com/deploy/srv-ct3k9...?key=...   # secreto: chame com POST para publicar

# Manual Deploy (botão)
Deploy latest commit · Clear build cache & deploy
```

Aprofundando: Render além do básico

Dica

Build estranho que funcionava antes? Use **Manual Deploy → Clear build cache & deploy**. O cache de camadas do Docker às vezes guarda uma dependência antiga.

Aprofundando: Render além do básico

Plano Free (referência)	Comportamento
Memória	512 MB
Inatividade	Dorme após 15 minutos sem requisições
Acordar	Cerca de 50 segundos
Horas por mês	750 por workspace (somando os serviços gratuitos)
Domínio próprio	Permitido, com HTTPS automático

Aprofundando: Render além do básico

1. Abra **Events** e encontre o deploy com erro.
2. Clique em **Logs** e procure, de baixo para cima, a primeira linha de erro (`ERROR`, `Caused by`).
3. Reproduza localmente com `docker build` e `docker run` usando as mesmas variáveis.
4. Corrija, faça push e acompanhe o novo deploy.
5. Se precisar do serviço no ar já, use **Rollback** para a última versão boa.

11

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Qual Language escolher no Render para Spring Boot?

A Java

B Docker

C Node

D Static

Quiz · questão 1 de 5

Resposta

Qual Language escolher no Render para Spring Boot?

A Java

✓ Docker

C Node

D Static

Por quê: O Render usa o Dockerfile do repositório.

Pergunta

Qual porta o Spring deve usar no Render?

A Sempre 8080

B A da variável PORT (`server.port=${PORT:8080}`)

C 80

D 443

Resposta

Qual porta o Spring deve usar no Render?

A. Sempre 8080

✓ A da variável PORT (`server.port=${PORT:8080}`)

C. 80

D. 443

Por quê: O Render define PORT=10000 por padrão.

Pergunta

Erro `failed to read dockerfile` geralmente é:

A. Falta de memória

B. Root Directory vazio quando o Dockerfile está em Back/

C. Banco offline

D. CORS

Resposta

Erro `failed to read dockerfile` geralmente é:

A Falta de memória

✓ Root Directory vazio quando o Dockerfile está em Back/

C Banco offline

D CORS

Por quê: Configure Root Directory = Back.

Pergunta

Para que serve o Health Check Path?

A Testar o CORS

B O Render só envia tráfego à nova versão quando ela responde

C Gerar logs

D Criar o banco

Resposta

Para que serve o Health Check Path?

A Testar o CORS

✓ O Render só envia tráfego à nova versão quando ela responde

C Gerar logs

D Criar o banco

Por quê: Se a nova versão falhar, a anterior continua no ar.

Pergunta

O que o `render.yaml` permite?

A. Criar o serviço a partir de um arquivo (Blueprint)

B. Configurar o Vercel

C. Rodar testes

D. Criar o cluster do Atlas

Resposta

O que o `render.yaml` permite?

✓ Criar o serviço a partir de um arquivo (Blueprint)

B Configurar o Vercel

C Rodar testes

D Criar o cluster do Atlas

Por quê: Segredos usam sync: false.

Fechamento

Resumo da aula

- ✓ **+ New → Web Service** → repositório → **Language: Docker**, **Root Directory: `Back`**, **Free**.
- ✓ Variáveis: `MONGODB_URI` ou `DB_URL/DB_USER/DB_PASSWORD`, `CORS_ORIGINS`, `JWT_SECRET`.
- ✓ `server.port=${PORT:8080}`: o Render define `PORT=10000`.
- ✓ Acompanhe em **Logs** até "*Your service is live*" e teste `/api/status`.
- ✓ Free dorme após 15 min (~50 s para acordar); `render.yaml` automatiza a criação.

Exercícios

1. Publique o seu back no Render e envie a URL do `/api/status`.

2. Configure o Health Check Path e faça um deploy com erro proposital: confirme que a versão anterior continua no ar.

3. Crie o `render.yaml` do seu projeto e recrie o serviço via Blueprint.

4. Meça o tempo de *cold start* depois de 20 minutos sem acessos.

Para ir além

- render.com/docs/web-services
- render.com/docs/docker
- render.com/docs/blueprint-spec

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 06?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web